Kristianstad University
SE-291 88 Kristianstad
+46 44-250 30 00
www.hkr.se

Independent project (degree project), 15 credits, for the Degree of
Bachelor of Science (180 credits) with a major in Computer Science
Spring Semester 2026
Faculty of Natural Sciences

Performance Trade-offs in API Rate Limiting: An Analytical and Empirical Study of Queue-on-Limit and Drop-on-Limit in a Node.js-Based API

Nicholas Malm, Jacob Hellgren

Authors

Nicholas Malm, Jacob Hellgren

Title

Performance Trade-offs in API Rate Limiting: An Analytical and Empirical Study of Queue-on-Limit and Drop-on-Limit in a Node.js-Based API.

Supervisor

Deema Aloom

Examiner

Kamilla Klonowska

Abstract

This study analyses performance differences between two rate-limiting strategies, Queue-on-Limit (QOL) and Drop-on-Limit (DOL), in a local API based on Node.js. The purpose is to investigate how these strategies affect P95 latency and the number of successful requests during steady and burst-load traffic. The study combines a systematic literature study, basic queueing theory, and a controlled experiment in a single-threaded environment.

The results show a clear trade-off between latency and the number of successful requests. QOL tends to give a higher success rate by queueing incoming requests; however, this leads to increased response time and a higher P95 latency when the system capacity approaches its limit. DOL performs at a lower and more stable latency by rejecting excess requests immediately at the cost of a decreased success rate.

The difference between the strategies is clear during high load and burst load conditions, resulting in queue buildup. These results can be understood by using queueing theory, where high request rates in relation to system capacity lead to increased response times. The study shows that neither strategy is inherently preferable; rather, the choice depends on whether low latency or high success rate is prioritized based on system requirements.

Keywords

API Performance, Rate Limiting Strategies, High-Load Conditions, Latency

Metrics

Acknowledgements

We would like to thank our supervisor, Deema Aloom, for her guidance and feedback throughout the thesis process. Her supervision helped us improve the work and keep the thesis focused.

Content

Dictionary

QOL (Queue-on-Limit) – A rate-limiting strategy that queues incoming requests exceeding a specified limit.

DOL (Drop-on-Limit) – A rate-limiting strategy that drops incoming requests exceeding a specified limit.

P95 (95th percentile latency) – The response time below which 95% of all requests are completed, used to capture tail latency.

M/M/1 – A basic queueing model describing a system with a single server, where arrivals and service times are random.

1 Introduction

1.1 Background

APIs play a central role in modern software systems, enabling communication between clients and services. When such systems are exposed to high loads or sudden traffic spikes, problems can occur that affect both performance and availability. If more requests arrive than the system has time to handle the response times can increase, and at the same time the risk of overload and instability becomes bigger.

To handle this kind of situation, rate limiting is often used, which means that the number of incoming requests is limited during a specific period. The purpose is to protect the system and create a more controlled usage of available resources. However, various strategies for rate limiting can have different consequences for how the system behaves during high load. One strategy can, for example, maintain low response times by quickly rejecting requests, while another instead tries to let more through by letting them wait in a queue. This means that trade-offs can emerge between, for example, latency, variation in response times, and the number of successful requests. It makes it relevant to study how different rate-limiting strategies affect a system's behavior during high load conditions.

1.2 Problem Statement

Rate limiting is commonly used in API gateways to protect the system from overload, ensuring reliability and stability during periods of high traffic. However, different rate-limiting strategies introduce different performance trade-offs. The Queue-on-Limit (QOL) strategy places incoming requests in a queue once the specified limit has been reached. While this may allow more requests to be processed successfully, it can also lead to increased response times and higher latency variability under high load. In contrast, the Drop-on-Limit (DOL) strategy immediately rejects new requests once the limit is exceeded. This can reduce latency, but results in a higher rate of failed requests.

Related studies have examined rate limiting primarily in distributed systems, while fewer studies have investigated how these strategies perform in a local single-threaded architecture. Because Node.js uses an event-driven, single-threaded architecture, these mechanisms may influence system performance differently compared to multi-threaded systems. Consequently, it remains unclear how QOL and DOL affect latency, response time variability, and request success rates under different load conditions in a local Node.js environment.

1.3 Research Questions

RQ1: What differences in P95 latency and request success rate emerge between Queue-on-Limit and Drop-on-Limit under increasing system load?

RQ2: How can the performance differences between Queue-on-Limit and Drop-on-Limit be explained in terms of queueing behavior under increasing load?

1.4 Scope and Limitations

This report examines how different rate-limiting strategies affect the way a system behaves during periods of high loads in a controlled local environment. The study focuses on an API implemented in Node.js that uses an event-driven and single-threaded architecture. Two rate-limiting strategies are compared: Queue-on-Limit and Drop-on-Limit. Using controlled experiments where load levels and load patterns vary, the strategies are analyzed to determine how they affect system performance. The analysis focuses on P95 latency, mean response time, variability in response times, and successful and unsuccessful requests.

The limitations of the study are that the experiments are conducted in an experimental environment and therefore do not represent a larger-scale production environment. Furthermore, only two strategies for rate limiting are evaluated. The traffic used in the experiments is synthetic, generated by a load generator, and does not correspond directly to real user traffic.

1.5 Research Gap

Earlier research shows that rate limiting and overload control are important ways of protecting systems during high load [1], [2], [3]. At the same time research mostly focuses on distributed environments, such as microservices, cloud systems, and API gateways. The performance is often evaluated here using metrics such as average latency, throughput, and stability [1], [4], [5]. Some studies mention tail latency and others that use queueing theory as analytical support, but these various perspectives are rarely combined in controlled comparisons of basic rate-limiting strategies.

According to Stack Overflow's Developer Survey 2025, Node.js is the most commonly used runtime environment [6]. The literature review conducted for this report found that, although Node.js is a common backend technology, most studies on rate limiting focus on larger distributed systems or microservices.

The reviewed articles do however give a limited insight into how the basic trade-off between Queue-on-Limit and Drop-on-Limit looks when the two strategies are isolated in the same, single-threaded Node.js environment [7]. This is relevant because related work from distributed, multi-threaded or more complex systems cannot automatically be assumed to apply to a simplified local API environment where the two strategies are isolated and compared during the same controlled conditions.

This study contributes with a controlled comparison of QOL and DOL by showing how the strategies differ in terms of P95 latency and request success rate as the load increases. The study especially clarifies during which conditions QOL gets less practical because of queue-buildup and how burst traffic can increase tail latency earlier than steady load. These insights are practically useful when developers need to choose between prioritizing predictable latency or processing more requests.

1.6 Thesis Structure

The thesis is structured as follows. Chapter 2 describes the method, systematic literature search, theoretical background, related work and experiment design. Chapter 3 presents the results from the experiment with steady load and burst load. Chapter 4 discusses the results in relation to the research questions, queueing theory, related work and the study's limitations. Chapter 5 summarizes the thesis conclusions and gives suggestions of future work.

1.7 Statement of Authorship & AI Use

We hereby confirm that this thesis is our own original work and has been carried out together, and we have contributed equally. AI-based tools were used only for grammar checking and language refinement, and not for the research, analysis, or conceptual development of the thesis.

2 Method

2.1 Research Design

This study was designed as a quantitative experimental comparative study. The purpose is to compare two rate limiting strategies, Queue-on-Limit and Drop-on-Limit, and how they affect performance metrics such as latency, response time variability and success rate during controlled high-load conditions in a local Node.js-based environment. The study combines a systematic literature search, basic queueing theory and a practical quantitative experiment.

The systematic literature search is used to find and identify previous research on the topic, and the theoretical background is used as analytical support to help us interpret the results. The empirical part consists of a controlled experiment where the two rate limiting strategies are compared during different load scenarios. In this way, the goal is not only to observe what happens, but also to create an understanding and explain why the differences between the strategies emerge.

2.2 Systematic Literature Search

The process of the systematic literature search started with selecting a relevant database. We chose the *Super Search* feature provided by Kristianstad University, which enables searches across multiple databases at the same time. The initial search gave us results primarily from *IEEE Xplore*.

After choosing a relevant database we used keywords related to API performance, Rate Limiting strategies, high-load conditions, and latency metrics. These keywords were then used in combination with synonyms and adjusted iteratively to create search strings in order to improve the relevance of the results. We filtered the results to peer-reviewed articles and published during the last five years. Depending on how many articles were shown in the results, the filters could be removed to see if there were additional articles older than five years that could be relevant.

When a search resulted in multiple articles, for example if the search was wide, we used a selection process where we first reviewed the title and could discard

some articles directly. After sorting out a few articles, we went further into the article and read the abstract from which we could further see if the article was relevant. The last step was to go into the full text. Using this process, the gathering of articles resulted in a first selection of twelve articles.

As the analytical approach in the thesis became clearer, the search was broadened with more conceptual search terms, for example tail latency, queueing theory, overload control and admission control. The purpose of this was to find studies that could strengthen the theoretical and analytical parts of this work, not just studies about API rate limiting. In addition to the first search, complementary search was therefore performed in ACM sources to find relevant journal articles outside IEEE. By performing this broader search, the literature base was expanded to a total of seventeen selected articles.

Because few articles addressed the exact comparison between Queue-on-Limit and Drop-on-Limit in a local Node.js environment, articles that were closely related to the study's area were also included. This included articles about rate-limiting, overload control, API performance, tail latency, and queueing theory, because these areas could contribute to understanding advantages, disadvantages and performance trade-offs between the strategies. Articles were excluded if they were too far from the study's focus area, for example if they did not concern computer science or system performance, focused on completely different types of systems, or if the implementation and context were too far away from API based load handling. By doing this, the literature was used both to identify related knowledge and to motivate the gap that this study investigates.

2.2.1 Searches

This subsection presents selected examples of searches that resulted in articles included in the final literature base.

Search example 1
Database: IEEE Xplore (Via HKR Super search)
Search string: ("API" OR "REST API" OR microservice*) AND ("rate limiting" OR "rate limit") AND (performance OR reliability OR latency)

Filters: Peer-reviewed, sorted by relevance

Total results: 22

Chosen: 6

Search example 2

Database: IEEE Xplore (Via HKR Super Search)

Search string: "queueing theory" AND ("web services" OR API) AND ("response time" OR throughput)

Filter: Peer-Reviewed, sorted by relevance

Total results: 10

Chosen: 1

2.3 Theoretical Background

Modern web-based APIs handle incoming requests from clients. Every request must be processed by the server before an answer can be sent back to the client. When the number of requests increases, the capacity of the server can be reached, which leads to more incoming requests than the server can handle.

As the number of incoming requests increases past the server's capacity, the rest of the requests are placed in a queue of waiting requests. The total response time of a request therefore consists of both the waiting time in the queue and the actual processing time. As the queue grows, the observed latency is therefore increased.

In queueing theory arrival rate (λ) describes the number of incoming requests per time unit, while service rate (μ) refers to how fast the system can handle requests. The relationship between these values shows how much of the system is utilized. This is defined as $\rho = \lambda/\mu$. As the utilization approaches 1, the system becomes increasingly saturated [8].

The M/M/1 model is a simple model used in queueing theory to analyze systems where tasks are processed by a single server. The first *M* means that arrivals happen randomly over time. The second *M* means that the service time is also random. The *1* means that the system has a single server for processing the requests or tasks. In the model, there is a server that handles the requests, a queue where incoming requests are waiting, and the requests are handled one at a time. The expected time in the system depends on the relation between the arrival rate (λ) and the service rate (μ). As mentioned before, the system becomes heavily loaded as the arrival rate approaches the service rate, which causes queues to build up and the waiting time to increase. This can be described using the formula:

$$W = \frac{1}{\mu - \lambda}$$

Where:

- **W** = expected time in the system

- λ = arrival rate
- μ = service rate [8]

In our study, the API server handles incoming requests through the Node.js event loop, where the requests are practically treated sequentially [7]. In this study, the API is interpreted as a simplified single-server system, where incoming requests can be queued as the load exceeds the system's capacity. From this perspective, queueing theory is used as an analytical tool to help explain how different rate-limiting strategies may affect latency behavior.

Queueing theory is only used here on a basic level as an analytical tool. The focus is on simple concepts from single server queues, such as arrival rate, service rate and queue buildup. This is to interpret how latency changes as the load increases. It is therefore not used as an exact model of Node.js, but instead as simplified analytical support to understand observed results.

In this study, two different strategies are used to handle situations where the load exceeds the server's capacity. One is the Queue-on-Limit strategy, which places incoming requests in a queue until the server can process it. The other is a Drop-on-Limit strategy, which instead rejects new requests once the cap has been reached, which limits the queue growth, but at the same time lowers the number of successful requests.

2.4 Related Work

2.4.1 Rate limiting and system performance

Related work can be divided into three groups, each having an overall theme. The first group is articles focused on rate limiting and system performance. Here, most studies emphasize that modern distributed systems, such as APIs, microservices, and cloud environments, are extremely vulnerable to overload. What they have in common is that they use traffic handling, especially rate limiting and load balancing, as the most important defense mechanism to prevent servers from crashing or resources running out, called resource starvation [1], [2], [4]. This

applies regardless of whether the overload depends on sudden, legitimate traffic spikes or intentional attacks such as DoS and DDoS [4], [9]. Another similarity between several of the articles is that they use related performance metrics to evaluate their solutions, for example, latency or response time, throughput, and different forms of success rate, failure rate, reliability, or accuracy [2], [4], [5]. However, the actual choice of metrics differs between studies depending on whether the focus lies on distributed rate limiting, attack mitigation, load balancing, or admission control. This suggests that the results of these studies are context-dependent, depending on the environment in which the experiments are conducted. By only focusing on metrics, major differences in system behavior may be overlooked.

The studies often combine rate limiting with other patterns, such as load balancing and request bundling, or build advanced rate limiters against attacks, with, for example, machine learning [1], [9]. While the solutions are impressive and large-scale, none of them make an isolated comparison of the basic strategies, Drop-on-Limit and Queue-on-Limit. This indicates a fundamental lack of understanding of the trade-off between a Queue-on-Limit and a Drop-on-Limit rate-limiting system, and how the comparison can help to understand how latency versus success rate is affected. This is where our study can contribute to explaining the trade-off between these basic strategies, in which more complex solutions are built.

The articles do have their differences, with one of the differences being in the choice of environment that the researchers chose to implement their rate limiting. The studies evaluate rate limiters in large, distributed environments, for example, with Redis databases, distributed load balancers, and Docker containers behind Nginx [4], [10]. However, none of the studies evaluate how the strategies work in single-threaded, event-driven environments such as Node.js. Because Node.js handles asynchronous requests and queues through its event loop, no assumptions can be made that results from distributed or multi-threaded systems apply. Furthermore, since Node.js has a single-threaded event loop, this may affect how queues work in comparison to a multi-threaded system.

2.4.2 Overload Control

The second theme the related work can be divided into is overload control, where the common focus of the articles is on how to protect microservices from crashing under extreme load. One thing the studies here have in common is that they focus on proactively dropping or rejecting incoming requests when the system approaches starvation, instead of just distributing the load. They do, however, implement these solutions in different ways. One study, for example, calculates if the request can be completed in time, and if not, it gets rejected instead [3]. In other studies, the decisions are instead based on the server's physical health, like CPU and memory utilization, or on how long the queue is at the recipient [11], [12]. This shows how much of the research is mainly focused on rejection-based rate-limiting strategies, and queueing-based rate-limiting is not as explored. This suggests how bias can affect how overload is handled. The studies do, however, not directly compare the two simple ways of handling overload in a single service: letting a request wait in a queue or rejecting it directly. Instead, they focus on rejecting the requests before it even reaches that stage. This indicates a trade-off that has not been properly emphasized in research.

Furthermore, related studies showed that there is an architectural variation in where the mechanisms, such as rate limiting and load control, are placed in the system. Some studies implement admission control on the gateway level, where incoming requests are evaluated against response time goals before they are allowed to pass [3]. In contrast to this, other studies distribute the control logic across decentralized components such as sidecar proxies (rate limiting, load balancing) [12]. However, what these studies have in common is that the load balancing happens on an infrastructural level in distributed, multithreaded systems. This indicates that the results of the studies may not be transferable to a single-threaded Node.js event loop.

2.4.3 Tail latency and queueing theory

The last group the articles can be divided into is the ones around tail latency and queueing theory. The studies here are implemented in large-scale environments such as Amazon and Uber, and what they have in common is that most of them

point out that not just the average latency, but also the tail latency can degrade user experience in systems [13], [14]. This reinforces our choice of tail latency as a performance metric. One thing to note here is that, as the studies are implemented in these large-scale environments with extreme distributed solutions, a possible gap in the research is that none of them investigates the most basic strategies.

From all the articles gathered in this area, we can also see clear differences in the environment they investigate and where they look for the problem. Some studies focus on large distributed systems, for example Amazon Web Services and Uber, where tail latency appears through network congestion, internal errors and complex chains of dependencies between services [13], [14]. Other studies are based on microservice environments where multiple services compete for the same resources [12]. Some studies build upon queueing theoretical models to mathematically describe the relation between arrival rate, service rate, and waiting time, instead of testing an actual system [15]. This shows that the articles differ in both the technical environment and the cause of tail latency, and in how the problem is solved.

Overall, related research shows that tail latency is an important metric to understand the performance problems in loaded systems, and that queueing theory is useful as an analytical support to explain how queue buildup affects latency. At the same time, these studies are primarily carried out in large distributed systems, or in more theoretical models, which makes it hard to directly transfer the result to a local, single-threaded Node.js environment. From this perspective, there is stick a lack of research that investigates how basic strategies such as Queue-on-Limit and Drop-on-Limit affect tail latency in a simpler and more controlled system.

2.5 Experimental Design

This experiment was designed to compare two rate limiting strategies in a controlled local Node.js-environment. By varying the load and traffic pattern, it was examined how the strategies affected performance measurements such as latency, success rate, and variation in response times.

2.5.1 Test Environment

The experiment was conducted using a local Node.js-based API, implementing rate-limiting middleware within the single-threaded event loop architecture. The experiment was conducted on a single machine to ensure consistency and reproducibility, and the same machine was used throughout the experiment. The test environment consisted of a computer with the Microsoft Windows 11 Home operating system, an AMD Ryzen 7 7700X processor with 8 cores and 16 logical processors, as well as 32 GB RAM.

2.5.2 Scenarios and Load Conditions

During the experiment, a custom-built load generator implemented in Python sent HTTP requests simulating high-load scenarios. The load generator supported both steady load and burst load scenarios. This made it possible to compare how the system behaves during more even load over time, and at shorter periods where the traffic suddenly increases.

During these scenarios two different rate limiting strategies were compared: Drop-on-Limit and Queue-on-Limit. The Drop-on-Limit Strategy rejects incoming requests exceeding a predefined threshold within a specified time window. This algorithm prevents queue buildup, maintaining low response times under high load conditions. In contrast, the Queue-on-Limit Strategy does not immediately reject requests exceeding the threshold. Instead, requests exceeding the limit are temporarily placed in a first-in, first-out (FIFO) queue and processed once system capacity becomes available. This algorithm may increase latency but can improve the request success rate.

The load level varied between different scenarios to examine how the strategies behave as the system goes from lower loads to levels closer to its capacity. The same API endpoint and the same workload were used in all scenarios to make the comparison as fair as possible. The endpoint used a simplified workload where every request waits for 100 ms before a response is sent back. This was done to ensure that the workload remained stable between all scenarios, and so that the comparison between QOL and DOL is easier to control. However, this workload

was simplified and does not represent all types of Node.js applications, such as endpoints with database calls, external API requests, or CPU-heavy processing. This limitation is further discussed in Section 4.5.

2.5.3 Variables

2.5.3.1 Independent Variables

Rate limiting strategy. Describes which of the two strategies is used in a specific test scenario. In this study, Drop-on-Limit and Queue-on-Limit are used.

Request load. Describes how many requests per second that are sent to the API by the load generator. By varying the request load, one can examine how the strategies behave when the system approaches saturation.

Load profile (steady load, burst load). Load profile describes what traffic pattern is used in a test scenario. Steady load means that requests are sent at a relatively even pace during the whole run. For example, if the requests are sent at 10 RPS, one request is sent approximately every 100 ms. Burst load instead means that the requests are sent in spikes, where more requests are sent during a shorter period before the load decreases again. This is used to see if the strategies are affected differently by a steady load compared to a more sudden increase in traffic.

2.5.3.2 Dependent Variables

Average response time (ms). Measures the average time from when a request is sent until a response is received. It gives an overall picture of how fast the system answers during the different test scenarios. It is used in combination with other measurements, as the average does not always capture variations or extreme values.

P95 latency. Shows the response time below which 95 percent of all requests are completed. It is used to capture tail behavior, which becomes very important during higher loads. Mean value can hide the slower parts of the response times, while P95 makes it clearer when a smaller number of requests get longer waiting times. Even if only 5% of all requests lie above the P95 value, this can still mean

many requests in a system with many users or frequent API calls. Shalev et al. also emphasize that tail latency events can be uncommon but still have a significant impact when they appear on a large scale [13].

Response time variability. Describes how much the response times vary between different requests. A system can have an acceptable average but still feel unstable if the response times vary greatly.

Success rate. The number of requests that are successfully processed in relation to the total number of requests sent. In the comparison of QOL and DOL, it is especially relevant as the strategies are expected to differ in how they let requests through.

Error rate. The number of requests that are not handled successfully in relation to the total number of requests, for example, when rejected by rate limiting.

2.5.4 Data Collection

For every request, data is collected about when the request was sent, when a response was received, and what status code was returned. From these timestamps, the response time is then calculated for each request. This data was used to calculate average response time, P95 latency, success rate, and response time variability.

To keep the results organized the data from each single run was saved separately, with filenames that automatically are marked with relevant information about scenarios, strategy and start time.

2.5.5 Statistical Validation

To decrease the risk of the results being affected by temporary variations, every scenario was run multiple times. For every single run the measurements were calculated separately, for example average response time, and P95 latency. These values were then compiled over multiple runs of the same scenario to give a more reliable picture of how stable the results were. This made it easier to assess whether observed differences were real and not just random deviations.

2.5.6 Experiment Overview

In the experiment, every combination was run by strategy, load profile, and request rate. The combination was also run five times each for statistical validation. Five runs were considered sufficient in this study because the experiment included 16 scenario combinations, based on two strategies, two load profiles, and four request rates. The results were therefore based on 80 runs in total. Since the experiment was carried out in a controlled local environment, with the same computer, endpoint, and workload in all scenarios, this was considered enough to compare average results across repeated runs and reduce the influence of temporary variations between individual runs. The results presented in Chapter 3, therefore, build on compilations over multiple runs instead of a single run. The scenario lasted for 30 seconds, and the same API endpoint was used throughout the experiment. The table below shows the experiment factors and metrics that were used.

Factor	**Values**
Strategies	Drop-on-Limit, Queue-on-Limit
Load profiles	Steady load, Burst load
Request rates	7, 9, 11, 13 RPS
Repeats	5 per scenario
Duration	30 seconds per run
Metrics	Success rate, error rate, mean latency, P95 latency, standard deviation

Table 1. Experiment Overview

The reported values for mean latency, P95 latency, and standard deviation were first calculated for every single run. Thereafter, the results were compiled by taking the mean value over the five runs in the same scenario. In this way, every row in the result tables represents a scenario instead of a single run.

3 Results

In this chapter, the results from the experiment are presented. The results are divided into two load profiles: steady and burst load. This is to make it clear how DOL compares to QOL during the different types of traffic. For every scenario, DOL and QOL are compared using the dependent variables success rate, error rate, mean latency, P95 latency, and variation in response time. The focus here lies in presenting what happened in the experiment.

3.1 Result Presentation

The results are presented by load profile and strategy. First, the steady load results are presented, followed by the burst load results. The chapter focuses on success rate, error rate, mean latency, P95 latency, and response time variability.

3.2 Steady Load Results

In this section, the results from the steady load scenarios are presented. Here, requests were sent at steady rates of 7, 9, 11, and 13 requests per second. The purpose was to see how Drop-on-Limit and Queue-on-Limit behave when the load increases gradually, but in this case without short traffic spikes.

Rate (RPS)	Strategy	Success rate (%)	Error rate (%)	Mean latency (ms)	P95 latency (ms)	SD (ms)
7	DOL	100.0	0.0	110.0	121.1	6.0
7	QOL	100.0	0.0	112.2	126.0	7.7
9	DOL	84.6	15.4	92.8	122.9	39.5
9	QOL	100.0	0.0	109.7	122.0	5.9
11	DOL	50.7	49.3	56.5	116.3	54.7
11	QOL	86.5	13.5	495.2	637.3	214.0
13	DOL	50.1	49.9	55.9	115.8	54.1
13	QOL	73.3	26.7	434.3	641.2	268.9

Table 2. Steady load results

At 11 and 13 RPS, the difference between the strategies became clearer. QOL still achieved a higher success rate than DOL, but its P95 latency increased

considerably, while DOL kept P95 latency relatively stable by rejecting a larger share of requests.

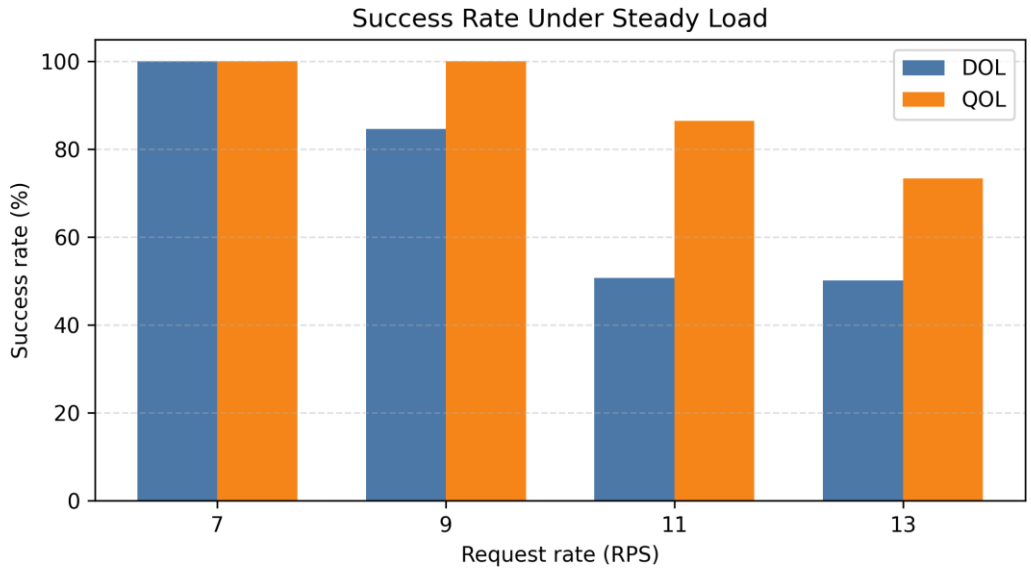

Figure 1. Success rate under steady load

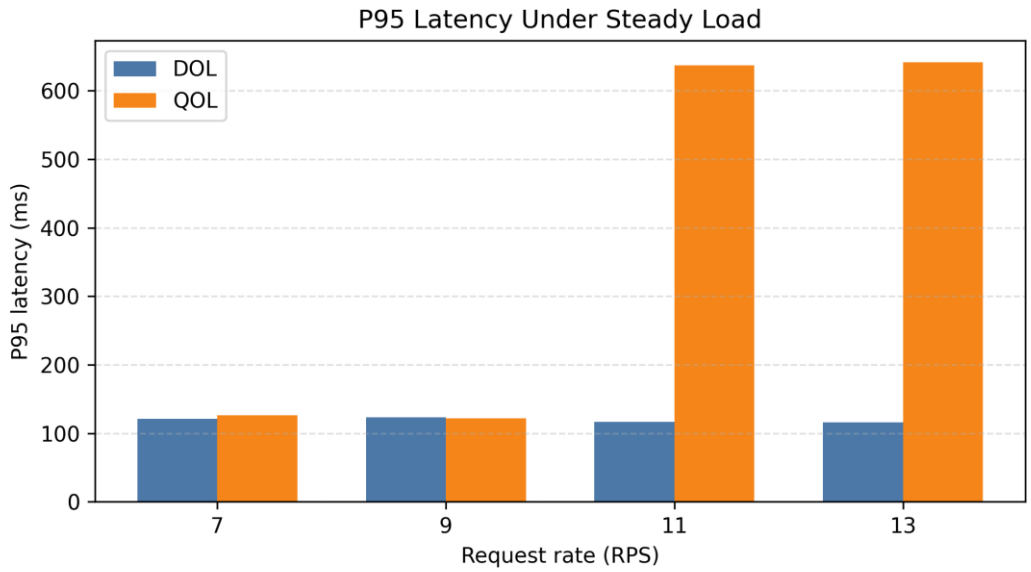

Figure 2. P95 latency under steady load

Figure 1 and Figure 2 show the difference between DOL and QOL under steady load. At lower load levels, both strategies perform similarly. Once the load increases, however, the difference becomes clearer. DOL keeps P95 latency

stable, but at the cost of more rejected requests. QOL accepts more requests, but has significantly higher P95 latency at the higher request rates.

3.3 Burst Load Results

This section covers the results of the burst load scenarios. Here, the requests were sent with burst traffic at rates of 7, 9, 11, and 13 requests per second, the same as for steady load. This was done to see how Drop-on-Limit and Queue-on-Limit act when the load increases in short traffic spikes instead of gradually.

Rate (RPS)	Strategy	Success rate (%)	Error rate (%)	Mean latency (ms)	P95 latency (ms)	SD (ms)
7	DOL	82.2	17.8	90.7	119.9	41.7
7	QOL	91.9	8.1	234.2	620.4	199.3
9	DOL	74.3	25.7	81.6	118.3	47.5
9	QOL	92.4	7.6	429.5	621.2	161.0
11	DOL	50.8	49.2	56.4	116.2	54.3
11	QOL	80.7	19.3	478.9	640.4	240.4
13	DOL	50.1	49.9	55.9	115.7	54.2
13	QOL	71.1	28.9	424.6	640.8	275.5

Table 3. Burst load results

The results show that both strategies experienced rejected requests already at 7 RPS. At 9 RPS, DOL rejected a larger portion of requests, while QOL maintained a higher success rate. At higher load levels, QOL continued to have a higher success rate than DOL. This also led to significantly higher latency and P95 latency.

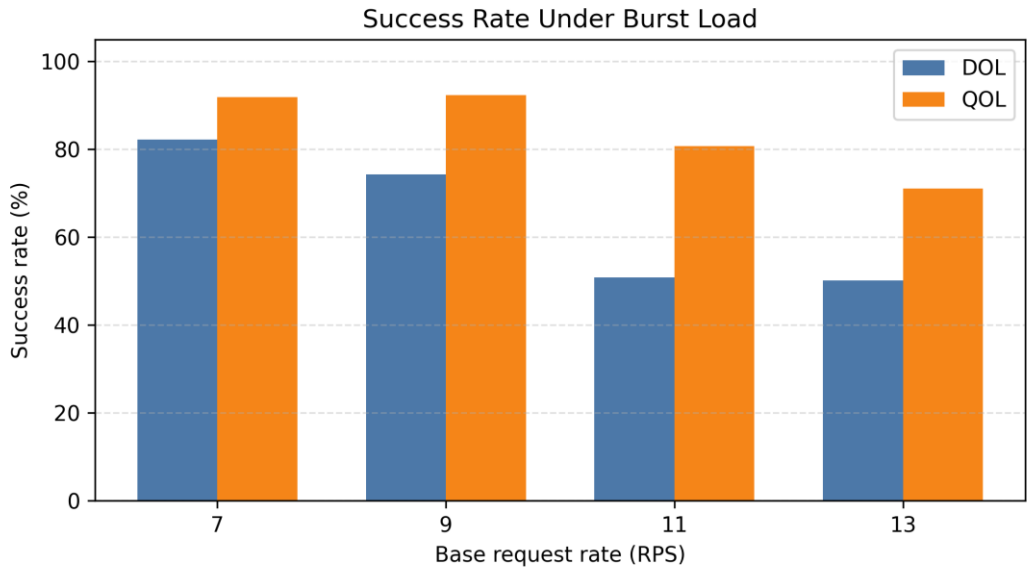

Figure 3. Success rate under burst load

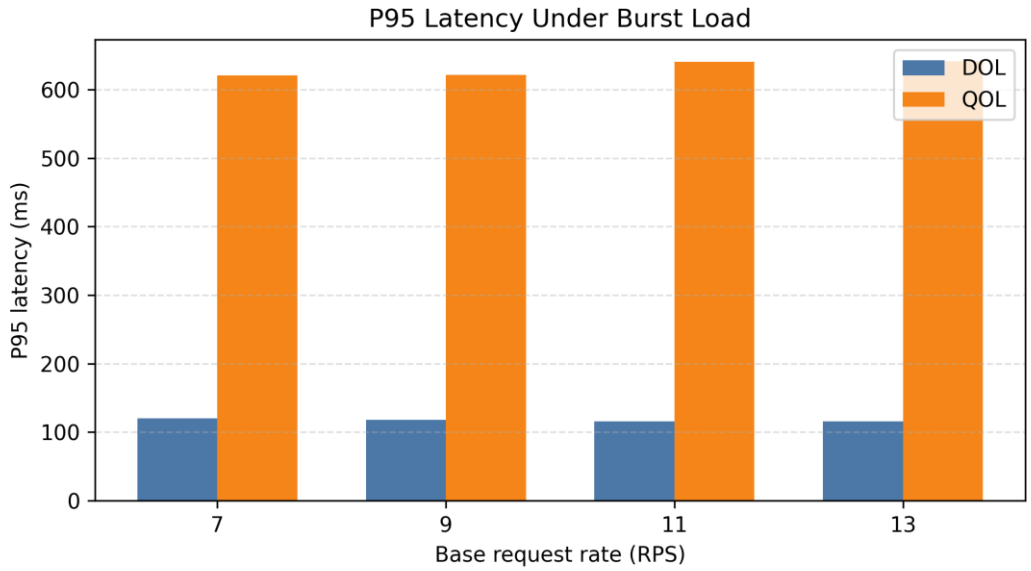

Figure 4. P95 latency under burst load

Figure 3 and Figure 4 show that the difference between DOL and QOL also appears under burst load. QOL generally achieves higher success rate than DOL. At the same time this leads to significantly higher P95 latency. Compared with steady load, the burst scenario shows the same trade-off, but earlier. QOL already had high P95 latency at 7 and 9 RPS, while DOL kept P95 latency more stable but

rejected more requests. This indicates that the difference between the strategies appears at lower base rates under burst traffic than under steady load.

4 Discussion

4.1 Interpretation of RQ1

The results show clear differences between QOL and DOL as the system load increases. Under low load conditions, both strategies behave similarly, with high success rates and low P95 latency. However, as the load approaches system capacity, a clear trade-off emerges between latency and request success rate.

In the steady load scenario, QOL achieves a higher success rate than DOL. This is because QOL allows incoming requests to be placed in a queue instead of being rejected immediately. However, this comes at the cost of increased latency. The P95 latency for QOL increases significantly, indicating that queued requests experience longer waiting times before being processed. In contrast, DOL maintains relatively stable and lower P95 latency by rejecting excess requests, although this results in a lower success rate.

This trade-off can be seen earlier under burst load. In a burst scenario, requests are not evenly distributed over time as during steady load. Instead, multiple requests can arrive in close proximity during a short period of time. For QOL, this means that the queue could fill up quicker, even when the average or basic request rate is relatively low. Once more requests must wait in the queue before being processed, the slowest requests get much longer response times. This explains why QOL had high P95 latency already at 7 and 9 RPS under burst load, while the same increase primarily was visible at higher request rates under steady load.

The results also show a limitation with QOL. The queue can only improve success rate if there is still space available for a waiting request. Once the queue is full, QOL can no longer handle overload by letting requests wait. Instead, it must start rejecting requests. This explains why QOL is more like DOL at higher sustained loads, even if QOL already has created longer waiting times for the requests that are placed in the queue.

4.2 Queueing Behavior and RQ2

The performance difference between QOL and DOL can be explained by how the strategies handle overload from a queueing perspective. In this study, QOL is implemented as a bounded first-in-first-out queue (FIFO), where a request is being processed while other requests are being placed in the queue.

This means that QOL does not eliminate overload, it delays it by allowing requests to wait before being processed. During increased load, more requests are coming in than the system can process. As QOL places requests in a queue instead of rejecting them, more requests are processed, and in turn, the success rate increases. However, this comes with a trade-off, as more requests are added to the queue, latency increases.

This queueing behavior becomes especially visible under burst load. Since requests can arrive closer together than during steady load, more requests can reach the system before earlier requests have been processed. In QOL, this leads to faster queue buildup, which increases the waiting time for the requests further back in the queue. Because P95 measures the slower requests, this queue buildup can be seen clearly as higher P95 latency. This explains why QOL is affected earlier under burst load than under steady load.

As mentioned earlier, the queue is finite and cannot indefinitely add new requests to the queue. When the queue has reached max capacity, QOL begins to behave more similarly to DOL and starts rejecting requests. This decreases the success rate of requests as the excessive requests are not being properly processed.

The differences between the two strategies become evident when the queue used in QOL is not at max capacity. When requests are still placed in the queue, the QOL strategy has a higher success rate, but with a trade-off of having a higher latency. In comparison, DOL has shown that it has a low latency but a lower success rate as well.

In this study, queueing is not measured directly. Instead, P95 latency and success rate are used to analyze the experimental results. Increased P95 latency indicates

longer queue waiting times, while higher success rates indicate that QOL processes requests that would otherwise be rejected.

4.3 Relation to Queueing Theory

The results can be connected to basic queueing theory. This is especially related to how fast requests arrive at the system and how fast the system can handle them. In the theory part, this was described as the arrival rate and service rate. When the number of incoming requests approaches or exceeds the system's capacity, requests start waiting, leading to higher response times. This can be further explained using queueing theory, where the arrival rate (λ) is approaching the system's service rate (μ). When λ approaches μ, the system becomes sensitive to smaller increases in load, quickly leading to increased response times and latency. This helps explain why the largest increase in QOL latency appears at the higher request rates, especially around 11 and 13 RPS.

This is most evident in Queue-on-Limit. Because QOL places requests in a queue instead of rejecting them immediately, more requests are allowed to be handled. At the same time, this means that some requests must wait before they get their responses. Therefore, latency, especially P95 latency, increases once the load increases. This matches queueing theory, where the waiting time increases as the system approaches its capacity.

Drop-on-Limit works differently. DOL limits queue buildup by rejecting requests once the limit is reached. This results in that fewer requests are handled in total, but the requests that are handled are not affected in the same way by queue waiting time. This explains why DOL in the results often has a lower P95 latency, but also lower success rate at higher loads.

The M/M/1 model is therefore not used as an exact model for the Node.js system, but as a simplified way of understanding the results. The experiment, for example, contains a limited queue and synthetic traffic, which means that the model can't predict exact latency values. However, the model does help explain why QOL

gets higher latency when more requests are waiting in queue, while DOL keeps latency lower by rejecting requests earlier.

In short, the results show the same basic patterns as queueing theory describes. As the system is loaded close to its capacity, queue buildup leads to longer waiting times. This supports the interpretation that the difference between QOL and DOL primarily is about a trade-off between higher success rate and higher P95 latency.

4.4 Comparison with Literature

As mentioned in Chapter 2.4, overload control is often described as a way to protect systems by limiting or rejecting requests once the load gets too high. The results in this study follow the same basic principle for Drop-on-Limit. DOL kept the P95 latency low, by rejecting requests once the limit was reached. At the same time this led to lower success rate at higher loads. This shows that rejection can protect response times, but at the cost of the number of requests that are handled.

The results for Queue-on-Limit can instead be connected to related work about queueing and tail latency. The related studies show that average latency is not always sufficient to describe the system's performance, because a smaller proportion of slower requests still can affect the overall performance. In this study, this can be seen by QOL often giving higher success rate than DOL, but at the same time much higher P95 latency once requests started queueing. This supports the idea that queueing can improve request completion but also increase tail latency.

Compared with much of the related work, this study uses a simpler and more isolated environment. Earlier studies often investigate distributed systems, API gateways, microservices, cloud environments or mechanisms for overload control in combination with other infrastructure components. This study, however, compares QOL and DOL in the same local Node.js-based API, which makes the basic trade-off between rejecting requests and queueing requests easier to observe. This means that the results should not be generalized directly to larger production

systems. They do, however, help explain the underlying performance trade-off that such systems also need to handle.

4.5 Threats to Validity

As the experiment was run in a local Node.js API with a single-threaded event loop, an external threat to validity is how well the results can be applied in other environments, such as more complex or distributed systems. The study also only compares Queue-on-Limit and Drop-on-Limit, which limits how well it can be generalized to other types of rate-limiting. The predetermined load scenarios and synthetic workload might also be a threat to validity, as they do not fully reflect real-world traffic in a production environment.

Because this study compares two different rate limiting strategies, which behave in different ways, their need for different logic might differ and that could lead to an internal threat to validity. If the implementations are not comparable at the same level, the results can be affected by the implementation rather than only by the strategies themselves. Another internal threat to validity is background processes. Even if other applications and programs on the experiment system are turned off, various background processes or temporary variations in the system performance might affect the results. To reduce the risk posed by these threats, the same endpoint and basic workload are used. The tested endpoint is also simplified as it only waits for 100 ms before returning a response. This improves the control between the runs but limits how well the results can be applied to endpoints with CPU-heavy work, database calls or external API requests, which could behave differently in a Node.js environment.

To get the most reliable results possible, each scenario is run multiple times, and the results are compiled over these runs. The results are useful as a controlled comparison between the strategies. However, they should not be interpreted as a full statistical generalization. In addition, the same hardware and environment are used throughout the experiment. Despite this, variations might still arise between the runs.

4.6 Practical Implications

The results of this study have several practical implications for the choice of rate-limiting strategies in API systems. The decision between QOL and DOL depends on which aspect of performance is prioritized, namely latency or request success rate. As shown in the results, QOL has a higher request success rate but at the cost of increased latency under higher load conditions, whereas DOL maintains lower latency but with a lower request success rate.

DOL is a suitable strategy when low latency is critical and when faster response times are more important than ensuring every request is successfully processed. This is particularly important in real-time systems like user-facing APIs. By rejecting excess requests immediately, DOL prevents queue buildup and maintains low latency.

In contrast, QOL is more suitable in situations where processing as many requests as possible is prioritized. By queueing excess requests, QOL can improve the overall request success rate. This makes QOL appropriate for systems where temporary delays are acceptable in exchange for higher completion rates.

However, QOL becomes less suitable under sustained high load conditions. During these conditions, queue buildup occurs quickly, leading to increased P95 latency. This is particularly evident at higher request rates, such as 11-13 RPS, where latency increases significantly. As a result, the effectiveness of queueing is reduced, and the strategy begins to behave more similarly to DOL, rejecting requests it cannot process. This may negatively impact user experience.

Overall, the results highlight a trade-off between latency and request success rate. QOL prioritizes higher completion rates by queueing requests, while DOL prioritizes low latency. Neither strategy is inherently preferable; the choice depends on system requirements and which trade-off has the greatest impact.

4.7 Social and Ethical Aspects

This study has some social relevance because many digital services use APIs that need to be fast and available. Rate limiting can therefore affect users, especially when a system gets higher loads. If a request is put in a queue, more requests can be processed, but the user may need to wait longer. If requests are denied, the response time can be kept lower, but some users may not get a successful response.

The results are especially interesting for developers and system owners who need to choose how an API should handle high loads. In a larger perspective, it is about how technical choices can affect availability, response time, and user experience.

The ethical risks in this study are small because no real users or personal information is used. However, rate limiting can have ethical aspects in real systems, because rejected or delayed requests can affect users differently depending on which service it concerns. Therefore, such solutions should be adapted to the purpose of the system and the needs of the user.

5 Conclusion

This study compared Queue-on-Limit and Drop-on-Limit in a controlled local Node.js-based API environment. The main finding is that the strategies show a clear trade-off between request success rate and P95 latency. QOL can process more requests by placing them in a queue; however, this leads to longer waiting times and higher P95 latency once the load approaches the system's capacity. DOL keeps latency more stable by rejecting requests earlier but therefore gets a lower success rate.

The study's contribution is that it isolates this trade-off between QOL and DOL in the same simplified experiment environment. The results show that QOL should not only be seen as a strategy for higher success rate, because queue buildup can make the strategy less practical under higher load and burst traffic. The main conclusion is therefore that the choice between QOL and DOL should be based on what the system prioritizes: stable and predictable latency or that more requests are processed.

DOL can be more suitable in systems where low and stable latency is most important, for example, user-friendly APIs where late responses quickly affect user experience. QOL can be more suitable when it is more important that more requests are processed and some waiting time is acceptable. Neither strategy is therefore always best. Their usability depends on system requirements and what trade-off is the most acceptable.

The study is limited because the experiment was implemented in a simplified local Node.js environment with a single endpoint and a fixed 100 ms workload. The results should therefore not be generalized directly to larger production systems. Future work should test the same strategies in more realistic systems, for example with database calls, external API requests, CPU-heavy tasks, different queue sizes, and longer testing periods.

6 References

[1] A. E. Malki and U. Zdun, “Combining API Patterns in Microservice Architectures: Performance and Reliability Analysis,” in *2023 IEEE International Conference on Web Services (ICWS)*, Jul. 2023, pp. 246–257. doi: 10.1109/ICWS60048.2023.00044.

[2] K. Caucidheesan and G. Poravi, “Analysing and Modelling the Accuracy and Latency Trade-offs in Rate Limiting on API-Gateway,” in *2025 International Research Conference on Smart Computing and Systems Engineering (SCSE)*, Apr. 2025, pp. 1–7. doi: 10.1109/SCSE65633.2025.11030994.

[3] H. Xu and J. A. Colmenares, “Admission Control with Response Time Objectives for Low-latency Online Data Systems,” Dec. 23, 2023, *arXiv*: arXiv:2312.15123. doi: 10.48550/arXiv.2312.15123.

[4] T. Kalyanasundaram, K. Panchalingam, T. Jegatheesan, A. Wijayasiri, and S. Perera, “Load Balancer Filter-Based Approach To Enable Distributed API Rate Limiting,” in *2025 37th Conference of Open Innovations Association (FRUCT)*, May 2025, pp. 75–85. doi: 10.23919/FRUCT65909.2025.11008311.

[5] A. E. Malki and U. Zdun, “Automated Pattern-Based Recommendation for Improving API Operation Performance and Reliability in Cloud-Based Architectures,” in *2023 IEEE International Conference on Software Services Engineering (SSE)*, Jul. 2023, pp. 80–88. doi: 10.1109/SSE60056.2023.00021.

[6] “Technology | 2025 Stack Overflow Developer Survey.” Accessed: Apr. 21, 2026. [Online]. Available: https://survey.stackoverflow.co/2025/technology/

[7] “Node.js — The Node.js Event Loop,” Nodejs. Accessed: Mar. 15, 2026. [Online]. Available: https://nodejs.org/en/learn/asynchronous-work/event-loop-timers-and-nexttick

[8] S. B. Gershwin, “Markov Processes and Queues.” MIT OpenCourseWare, 2016. Accessed: Mar. 15, 2026. [Online]. Available: https://ocw.mit.edu/courses/2-854-introduction-to-manufacturing-systems-fall-2016/5d3e9532c2ff6e156d48e41f9bd9576f_MIT2_854F16_Queueing.pdf

[9] M. N. Alenezi, M. J. Reed, and M. A. Alhomidi, “Selective Windowed Rate Limiting for DoS Mitigation,” in *2017 9th IEEE-GCC Conference and Exhibition (GCCCE)*, May 2017, pp. 1–6. doi: 10.1109/IEEEGCC.2017.8448019.

[10] E. Mabotha, N. E. Mabunda, and A. Ali, “Performance Evaluation of a Dynamic RESTful API Using FastAPI, Docker and Nginx,” in *2024 Global Energy Conference (GEC)*, Dec. 2024, pp. 174–181. doi: 10.1109/GEC61857.2024.10881712.

[11] S. R. Boyapati and C. Szabo, “Self-adaptation in Microservice Architectures: A Case Study,” in *2022 26th International Conference on Engineering of Complex Computer Systems (ICECCS)*, Mar. 2022, pp. 42–51. doi: 10.1109/ICECCS54210.2022.00014.

[12] R. Bhattacharya, Y. Gao, and T. Wood, “Dynamically Balancing Load with Overload Control for Microservices,” *ACM Trans Auton Adapt Syst*, vol. 19, no. 4, p. 22:1-22:23, Nov. 2024, doi: 10.1145/3676167.

[13] L. Shalev *et al.*, “The Tail at Amazon Web Services Scale,” *IEEE Micro*, vol. 44, no. 5, pp. 23–29, Sep. 2024, doi: 10.1109/MM.2024.3420070.

[14] I.-T. A. Lee, Z. Zhang, A. Parwal, and M. Chabbi, “The Tale of Errors in Microservices,” *Proc ACM Meas Anal Comput Syst*, vol. 8, no. 3, p. 46:1-46:36, Dec. 2024, doi: 10.1145/3700436.

[15] M. shahooth Hamadi *et al.*, “Mapping the Characteristics of Web Services to Queueing Theory Models for Performance Evaluation,” in *2025 3rd International Conference on Business Analytics for Technology and Security (ICBATS)*, May 2025, pp. 1–8. doi: 10.1109/ICBATS66542.2025.11258372.